ASSIGNMENT- 01

Bachelor of Computer Applications (Hons. AI & ML)
Semester- III

Subject: Database Management System

Topic: Identifying Tables for a Real-World Application

By

Name: Akash P
Reg. No.: 2511021240008

Faculty Name: RENITA JOY
Faculty Signature: _____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
September 2026

DBMS ASSIGNMENT

Akash P
Database Design for a Real-World Application
Alliance University | Department of Computer Application

1. Online Food Delivery System

Application Name: Online Food Delivery System (modeled on platforms such as Swiggy or Zomato)

Purpose: The application allows registered customers to browse restaurants near them, view menus, place food orders, make payments online, and get the order delivered to a chosen address by a delivery agent. Restaurant owners can manage their menu items, and the platform tracks the entire order lifecycle - from order placement, to payment, to assignment of a delivery agent, to final delivery - along with a post-delivery rating and review submitted by the customer. The underlying database must therefore store information about customers, restaurants, menus, orders, payments, delivery agents, and reviews, while keeping the data consistent and avoiding unnecessary repetition.

2. List of Required Tables

#	Table Name	One line Role
1	Customers	Stores registered customer details
2	Addresses	Stores one or more delivery addresses per customer
3	Restaurants	Stores details of partner restaurants
4	MenuItems	Stores food items offered by each restaurant
5	Orders	Stores each order placed by a customer
6	OrderItems	Stores the individual menu items within an order
7	DeliveryAgents	Stores details of delivery personnel
8	Deliveries	Tracks assignment and status of an order's delivery
9	Payments	Stores payment details for each order
10	Reviews	Stores customer feedback for an order

Each table above serves a distinct purpose in the order lifecycle; none is included merely to increase count (for example, Addresses and OrderItems exist because a customer can have multiple delivery addresses and an order can contain multiple menu items, which cannot be represented cleanly inside a single row of Customers or Orders).

3. Table Structures (Columns / Attributes) and Relationships

3.1 Customers

Stores the profile information of every person who registers on the platform to order food. This is the central table that other activities (orders, addresses, reviews) are linked back to.

Column	Data Type	Key	Description
customer_id	INT	PK	Unique identifier for each customer
full_name	VARCHAR(100)	-	Name of the customer
email	VARCHAR(100)	UNIQUE	Email used for login and order updates
phone_number	VARCHAR(15)	UNIQUE	Contact number of the customer
password_hash	VARCHAR(255)	-	Encrypted login password
registration_date	DATE	-	Date the customer signed up on the platform

3.2 Addresses

Stores the delivery addresses saved by a customer. A customer may have more than one address (Home, Work, etc.), so this is kept as a separate table rather than as columns inside Customers.

Column	Data Type	Key	Description
address_id	INT	PK	Unique identifier for each address
customer_id	INT	FK	References Customers.customer_id - owner of the address
address_line	VARCHAR(150)	-	House/street/locality details
city	VARCHAR(50)	-	City of the address
state	VARCHAR(50)	-	State of the address
pincode	VARCHAR(10)	-	Postal code of the address
address_type	VARCHAR(20)	-	Label such as Home, Work, or Other

Relationship: One Customer can have Many Addresses (1:M).

3.3 Restaurants

Stores details of every restaurant that has partnered with the platform to sell food through it.

Column	Data Type	Key	Description
restaurant_id	INT	PK	Unique identifier for each restaurant
restaurant_name	VARCHAR(100)	-	Name of the restaurant
cuisine_type	VARCHAR(50)	-	Type of cuisine offered, e.g., North Indian, Chinese
address	VARCHAR(150)	-	Physical location of the restaurant
phone_number	VARCHAR(15)	-	Contact number of the restaurant
rating	DECIMAL(2,1)	-	Average customer rating of the restaurant

Column	Data Type	Key	Description
is_active	BOOLEAN	-	Whether the restaurant is currently accepting orders

3.4 MenuItems

Stores the individual food items that a restaurant offers, along with their price and category, so that customers can browse and order from a restaurant's menu.

Column	Data Type	Key	Description
item_id	INT	PK	Unique identifier for each menu item
restaurant_id	INT	FK	References Restaurants.restaurant_id - owning restaurant
item_name	VARCHAR(100)	-	Name of the food item
description	VARCHAR(200)	-	Short description of the item
price	DECIMAL(8,2)	-	Price of one unit of the item
category	VARCHAR(30)	-	e.g., Starter, Main Course, Dessert, Beverage
is_available	BOOLEAN	-	Whether the item is currently available for ordering

Relationship: One Restaurant can have Many MenuItems (1:M).

3.5 Orders

Stores one record for every order placed by a customer. It captures which customer ordered from which restaurant, to which address, and the overall status and value of the order.

Column	Data Type	Key	Description
order_id	INT	PK	Unique identifier for each order
customer_id	INT	FK	References Customers.customer_id - who placed the order
restaurant_id	INT	FK	References Restaurants.restaurant_id - source of the order
address_id	INT	FK	References Addresses.address_id - delivery address for the order
order_date	DATETIME	-	Date and time the order was placed
order_status	VARCHAR(20)	-	Placed, Preparing, Out for Delivery, Delivered, or Cancelled
total_amount	DECIMAL(8,2)	-	Total bill amount for the order

Relationships: One Customer can place Many Orders (1:M); One Restaurant can receive Many Orders (1:M); One Address can be used for Many Orders (1:M).

3.6 OrderItems

A linking table that stores which menu items (and how many of each) were part of a specific order. It is needed because a single order usually contains multiple menu items, and a menu item can appear in many different orders - a Many-to-Many relationship that this table resolves.

Column	Data Type	Key	Description
order_item_id	INT	PK	Unique identifier for each order line
order_id	INT	FK	References Orders.order_id - the order this line belongs to
item_id	INT	FK	References MenuItems.item_id - the item ordered
quantity	INT	-	Number of units of the item ordered
price_at_order	DECIMAL(8,2)	-	Price of the item at the time of ordering

Relationships: One Order can have Many OrderItems (1:M); One MenuItem can appear in Many OrderItems (1:M). Together, Orders and MenuItems share a Many-to-Many relationship through this table.

3.7 DeliveryAgents

Stores details of the delivery personnel who pick up orders from restaurants and deliver them to customers.

Column	Data Type	Key	Description
agent_id	INT	PK	Unique identifier for each delivery agent
agent_name	VARCHAR(100)	-	Name of the delivery agent
phone_number	VARCHAR(15)	-	Contact number of the agent
vehicle_number	VARCHAR(20)	-	Registration number of the delivery vehicle
is_available	BOOLEAN	-	Whether the agent is free to accept a new delivery

3.8 Deliveries

Tracks the delivery of a specific order: which agent was assigned, when the order was picked up and delivered, and the current delivery status. Kept separate from Orders because delivery timing and status are tracked independently of the order and payment status.

Column	Data Type	Key	Description
delivery_id	INT	PK	Unique identifier for each delivery record
order_id	INT	FK	References Orders.order_id - the order being delivered
agent_id	INT	FK	References DeliveryAgents.agent_id - assigned delivery agent
pickup_time	DATETIME	-	Time the order was picked up from the restaurant
delivery_time	DATETIME	-	Time the order was delivered to the customer
delivery_status	VARCHAR(20)	-	Assigned, Picked Up, Delivered, or Failed

Relationships: One Order has exactly One Delivery record (1:1); One DeliveryAgent can handle Many Deliveries over time (1:M).

3.9 Payments

Stores the payment details for each order, including the method used and whether the payment succeeded.

Column	Data Type	Key	Description
payment_id	INT	PK	Unique identifier for each payment
order_id	INT	FK	References Orders.order_id - the order being paid for
amount	DECIMAL(8,2)	-	Amount paid
payment_method	VARCHAR(20)	-	UPI, Card, Net Banking, or Cash on Delivery
payment_status	VARCHAR(20)	-	Pending, Successful, or Failed
payment_date	DATETIME	-	Date and time the payment was made

Relationship: One Order has exactly One Payment record (1:1).

3.10 Reviews

Stores the feedback a customer leaves after an order is delivered, covering both the restaurant's food and the delivery experience.

Column	Data Type	Key	Description
review_id	INT	PK	Unique identifier for each review
order_id	INT	FK	References Orders.order_id - the order being reviewed
customer_id	INT	FK	References Customers.customer_id - who wrote the review
restaurant_rating	INT	-	Rating given to the food/restaurant (1 to 5)
delivery_rating	INT	-	Rating given to the delivery experience (1 to 5)
comment	VARCHAR(250)	-	Optional written feedback from the customer
review_date	DATETIME	-	Date and time the review was submitted

Relationships: One Order has exactly One Review (1:1); One Customer can write Many Reviews across different orders (1:M).

4. Overall Relationship Summary

Parent Table	Child Table	Relationship	Reason
Customers	Addresses	1 : M	A customer can save multiple delivery addresses
Customers	Orders	1 : M	A customer can place many orders over time
Customers	Reviews	1 : M	A customer can review many of their past orders
Restaurants	MenuItems	1 : M	A restaurant offers many food items
Restaurants	Orders	1 : M	A restaurant receives many orders from customers
Addresses	Orders	1 : M	One saved address can be reused for many orders
Orders	OrderItems	1 : M	An order can include several menu items
MenuItems	OrderItems	1 : M	A menu item can be ordered in many separate orders
Orders	Deliveries	1 : 1	Each order has one corresponding delivery record
DeliveryAgents	Deliveries	1 : M	An agent handles many deliveries over time
Orders	Payments	1 : 1	Each order has one corresponding payment record
Orders	Reviews	1 : 1	Each order can receive one review from the customer

5. Conclusion

The ten tables above together model the complete lifecycle of an online food order: registering customers and their addresses, listing restaurants and their menus, capturing what was ordered, and tracking payment, delivery, and post-delivery feedback. Each table has a single, clear responsibility, foreign keys are used to connect related tables instead of duplicating data, and the Many-to-Many relationship between Orders and MenuItems is correctly resolved through the OrderItems linking table. This demonstrates how a real-world application such as a food delivery platform can be represented in a normalized relational database.